

Introduction to Apache Airflow

What is Airflow?

Apache Airflow is an open-source platform used to programmatically author, schedule, and monitor workflows. Think of it as a sophisticated system for managing and automating complex computational tasks and data processing pipelines.

- **Key Idea:** Instead of using cron jobs (which are good for simple, linear tasks but hard to manage for dependencies), Airflow allows you to define workflows as code (Python). This makes them:
 - **Dynamic:** Can generate pipelines dynamically.
 - **Scalable:** Can handle complex task dependencies.
 - **Maintainable:** Can be version-controlled (e.g., with Git), tested, and collaborated on.
- **It is NOT a data processing framework.** It doesn't process data itself (like Spark or Pandas). It is an **orchestrator**—it tells other systems *when* and *how* to process data.

Core Components

The Airflow UI and functionality are built on these main components:

1. **Web Server:** A Flask application that provides the user interface to inspect, trigger, and debug the behaviour of your workflows (DAGs).
2. **Scheduler:** The brain of Airflow. It is responsible for scheduling jobs (DAGs). It checks the DAGs folder, parses them, and decides what tasks need to be run, when, and in what order.
3. **Executor:** The mechanism *inside the scheduler* that actually runs the tasks. The default is a SequentialExecutor, but for production you use more powerful ones like CeleryExecutor or KubernetesExecutor.
4. **Metadata Database:** A SQL database (usually PostgreSQL or MySQL) where Airflow stores information about the state of DAGs, tasks, variables, connections, and more. The Scheduler, Web Server, and Executor all interact with this database.
5. **Worker:** Processes that execute the tasks assigned by the scheduler. (Used in distributed architectures with the CeleryExecutor).

Core Concepts

- **DAG (Directed Acyclic Graph):** The central concept. It's a collection of all the tasks you want to run, organized to show their relationships and dependencies. "Directed" means tasks have a direction (dependencies), and "Acyclic" means there can be no loops (a task can't lead back to itself).
 - **DAG Run:** An instance of a DAG being executed for a specific logical date/time.
- **Operator:** A template for a single task. It describes what *actually* gets done. Airflow has many built-in operators:
 - **BashOperator:** Executes a Bash command.

- PythonOperator: Calls a Python function.
- EmailOperator: Sends an email.
- SimpleHttpOperator: Sends an HTTP request.
- Many provider packages exist for other tools (e.g., SnowflakeOperator, S3FileTransformOperator).
- **Task:** An instance of an Operator. It's a parameterized operator that becomes a node in the DAG.
- **Task Instance:** A specific run of a task for a specific DAG Run. It has a state (e.g., success, running, failed, skipped, up_for_retry).
- **Sensor:** A special type of Operator that waits for a certain condition to be true (e.g., a file to arrive in a folder, a partition to appear in Hive, a time of day) before succeeding and letting downstream tasks run.

More about Airflow

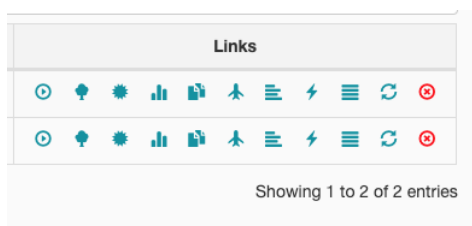
- **Workflows as Code (Python):** The biggest advantage. You define your DAGs in Python files. This gives you the full power of Python: loops, conditionals, dynamic DAG generation, etc.
- **Rich UI:** Provides deep insights into your pipeline execution, logs, task duration, Gantt charts, and more.
- **Extensibility:** You can create your own custom Operators, Hooks (connections to external platforms), and Sensors.
- **Community:** Huge and active community with many ready-to-use providers for almost any service (AWS, GCP, Azure, Snowflake, Databricks, etc.).

The Different Architectures

How you deploy Airflow depends on your needs.

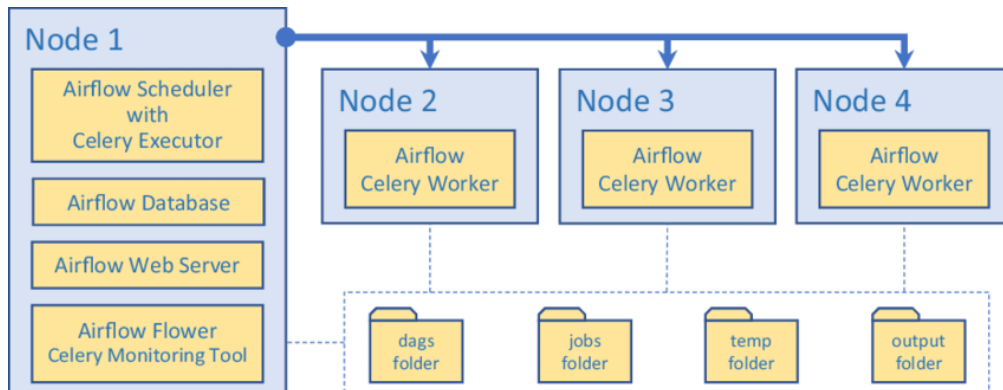
1. Single-Node Architecture (Standalone):

- All components (Scheduler, Web Server, Executor, Database) run on a single machine.
- Good for learning and development.
- **Executor:** Usually SequentialExecutor (runs one task at a time) or LocalExecutor (can run multiple tasks in parallel on the same machine).



Multi-Node (Distributed) - CeleryExecutor:

- The **Web Server** and **Scheduler** run on one master node.
- The **Metadata Database** runs on its own dedicated server (e.g., PostgreSQL).
- Multiple **Worker** nodes run separately. They are managed by a **message queue** (like Redis or RabbitMQ).
- The Scheduler adds tasks to the queue, and any available worker can pick them up and execute them. This allows for horizontal scaling.



2. KubernetesExecutor:

- The most dynamic and cloud-native approach.
- The Scheduler and Web Server run as pods in a Kubernetes cluster.
- For *each task*, the Scheduler asks the Kubernetes API to launch a brand new pod specifically for that task. The pod is destroyed when the task is complete.
- Highly efficient and isolated, but more complex to set up.

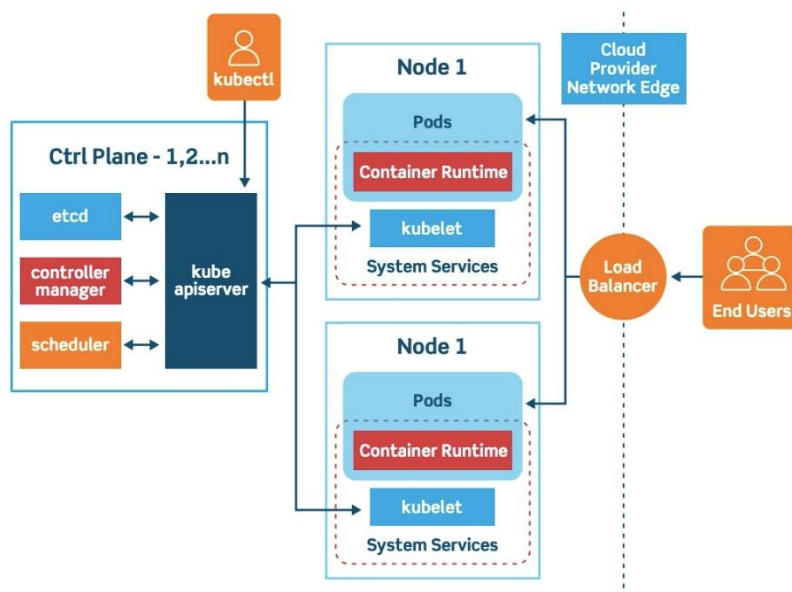


Diagram of a Kubernetes cluster with pods for Scheduler and Web Server. A separate area shows a series of short-lived pods popping up and disappearing for each task execution.

How does it work? (The Lifecycle of a DAG Run)

1. **Authoring:** You write a Python file defining your DAG and its tasks and save it in the DAGS_FOLDER.
2. **Parsing:** The Scheduler periodically scans the DAGS_FOLDER, imports the Python files, and parses the DAGs to build their structure in the Metadata Database.
3. **Scheduling:** For each DAG, the Scheduler checks its schedule (e.g., @daily) and creates a DAG Run instance for each interval that has passed and needs to be processed. The DAG Run is set to a state (e.g., running).
4. **Task Queuing:** The Scheduler examines the DAG's tasks. If a task's dependencies (e.g., task1 >> task2) are met, the Scheduler changes the Task Instance's state to queued.
5. **Executing:** The Executor (e.g., CeleryExecutor) sees the queued task and sends it to the message queue. An available Worker picks it up from the queue, changes its state to running, and executes it.
6. **Completion:** After execution, the Worker updates the Task Instance's state in the Metadata Database (success, failed).
7. **Monitoring:** The Web Server continuously reads from the Metadata Database to display the status of DAGs and tasks in the UI. You can see logs, retry failed tasks, and get alerts.

Installing Apache Airflow

The Old Way (Direct PIP install):

- pip install apache-airflow
- Then you had to manually initialize the database (airflow db init), create a user, and start the webserver and scheduler.
- **Problem:** Dependency conflicts and system-specific issues were very common. Not recommended for new users.

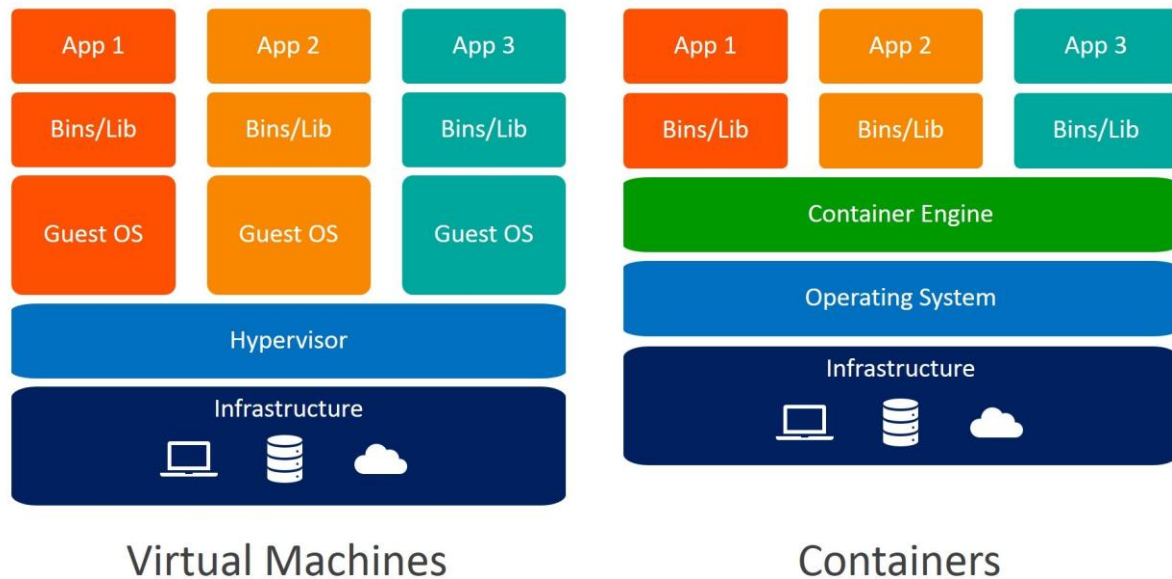
The Modern Way (Using Docker & Docker-Compose):

- This is the official and recommended method for development and production. It provides an isolated, reproducible environment.

8What is Docker?

- **Analogy:** A shipping container for software.
- It allows you to package an application with all of its dependencies (libraries, code, runtime) into a standardized unit called a **container**.

- This container can run reliably and consistently on any machine that has Docker installed, regardless of the underlying OS.
- **Key Terms:**
 - **Docker Image:** A read-only template with instructions for creating a container (like a class in OOP).
 - **Docker Container:** A runnable instance of an image (like an object instance).
 - **Dockerfile:** A text document with all the commands to assemble an image.



The docker-compose file

- A tool for defining and running multi-container Docker applications.
- You configure your application's services (containers) in a YAML file called docker-compose.yaml.
- Instead of running complex docker run commands with many flags, you just run docker-compose up.
- **The Official Airflow** docker-compose.yaml:
 - Apache Airflow provides an official docker-compose.yaml file that defines services for:
 - airflow-scheduler
 - airflow-webserver
 - airflow-worker (if using CeleryExecutor)
 - airflow-triggerer (for deferred operators)
 - postgres (or redis for the queue)
 - flower (a tool for monitoring Celery workers)

- It also handles volume mounts (to make your dags/, logs/, and plugins/ folders accessible inside the containers) and environment variables.

Example of a key section:

yaml

```
# This section defines the webserver container

x-airflow-common:
  <<: *airflow-common
  image: ${AIRFLOW_IMAGE_NAME:-apache/airflow:2.8.1}
  environment:
    - AIRFLOW__CORE__EXECUTOR=LocalExecutor
    -
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres/airflow
  volumes:
    - ./dags:/opt/airflow/dags # Mounts local dags folder
    - ./logs:/opt/airflow/logs # Mounts local logs folder
  depends_on:
    - postgres

services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow

  airflow-webserver:
    <<: *airflow-common
    command: webserver
    ports:
      - "8080:8080" # Exposes port 8080 on your local machine

  airflow-scheduler:
    <<: *airflow-common
    command: scheduler
```

Installation Steps with Docker:

1. Install Docker Desktop on your machine.
2. Download the official docker-compose.yaml from the Airflow website.
3. Create the necessary folders (dags, logs, plugins).
4. Run `docker-compose up -d` in the terminal from the folder containing the YAML file.
5. Open your browser and go to `http://localhost:8080`. The default login is `airflow / airflow`.



DAGs

All 53		Active 2		Paused 44		Running 7		Failed 1		Filter DAGs by tag		Search DAGs		Auto-refresh			
DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks										Actions	
dataset_consumes_1 consumes dataset-scheduled	airflow		Dataset	2024-03-21, 14:15:47	On s3://dag1/output_1.txt												
dataset_consumes_1_and_2 consumes dataset-scheduled	airflow		Dataset		0 of 2 datasets updated												
dataset_consumes_1_never_scheduled consumes dataset-scheduled	airflow		Dataset		0 of 2 datasets updated												
dataset_consumes_unknown_never_scheduled dataset-scheduled	airflow		Dataset		0 of 2 datasets updated												
dataset_produces_1 dataset-scheduled produces	airflow		@daily	2024-03-20, 00:00:00													
dataset_produces_2 dataset-scheduled produces	airflow		None														
example_branch_operator example examples2	airflow		00***	2024-03-20, 00:00:00	2024-03-21, 00:00:00												
example_branch_datetime_operator example	airflow		@daily	2024-03-20, 00:00:00	2024-03-21, 00:00:00												
example_branch_datetime_operator_2 example	airflow		@daily	2024-03-20, 00:00:00	2024-03-21, 00:00:00												
example_branch_datetime_operator_3 example	airflow		@daily	2024-03-20, 00:00:00	2024-03-21, 00:00:00												
example_branch_dop_operator_v3 example	airflow		*/1 * * * *	2024-03-21, 14:16:00	2024-03-21, 14:15:00												
example_branch_labels	airflow		@daily	2024-03-20, 00:00:00	2024-03-21, 00:00:00												